European Journal of Operational Research 221 (2012) 513–520

Contents lists available at SciVerse ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/ejor

Discrete Optimization

A simple and effective metaheuristic for the Minimum Latency Problem

Marcos Melo Silva [a,*], Anand Subramanian [a,b], Thibaut Vidal [c,d], Luiz Satoru Ochi [a]

[a] Universidade Federal Fluminense, Instituto de Computação, Rua Passo da Pátria 156, Bloco E – 3º andar, São Domingos Niterói-RJ 24210-240, Brazil
[b] Universidade Federal da Paraíba, Departamento de Engenharia de Produção, Centro de Tecnologia, Campus I – Bloco G, Cidade Universitária, João Pessoa-PB 58051-970, Brazil
[c] CIRRELT, Département d'informatique et de recherche opérationnelle, Université de Montréal, Montréal, Canada H3C 3J7
[d] Institut Charles Delaunay, Université de Technologie de Troyes, 10010 Troyes, France

ARTICLE INFO

Article history:
Received 15 September 2011
Accepted 26 March 2012
Available online 16 April 2012

Keywords:
Metaheuristics
Minimum Latency Problem
GRASP
Iterated Local Search

ABSTRACT

The Minimum Latency Problem (MLP) is a variant of the Traveling Salesman Problem which aims to minimize the sum of arrival times at vertices. The problem arises in a number of practical applications such as logistics for relief supply, scheduling and data retrieval in computer networks. This paper introduces a simple metaheuristic for the MLP, based on a greedy randomized approach for solution construction and iterated variable neighborhood descent with random neighborhood ordering for solution improvement. Extensive computational experiments on nine sets of benchmark instances involving up to 1000 customers demonstrate the good performance of the method, which yields solutions of higher quality in less computational time when compared to the current best approaches from the literature. Optimal solutions, known for problems with up to 50 customers, are also systematically obtained in a fraction of seconds.

© 2012 Elsevier B.V. All rights reserved.

1. Introduction

The Minimum Latency Problem (MLP) can be defined as follows. Let $G = (V, A)$ be a complete directed graph, where $V = \{0, \ldots, n\}$ is the set of vertices and $A = \{(i, j): i, j \in V, i \neq j\}$ is the set of arcs with associated travel time t_{ij}. Vertex 0 stand for the depot while other vertices represent customers. The MLP aims at finding a Hamiltonian circuit that minimizes $\sum_{i=0}^{n} l(i)$, $l(i)$ representing the latency of a vertex $i \in V$, that is, the total travel time to reach i. We also consider the additional constraint that the circuit must start and end at vertex 0 and also that $l(0) = 0$. The MLP is a variant of the well-known Traveling Salesman Problem (TSP) and it is known in the literature under various other names: *Traveling Repairman Problem* (Tsitsiklis, 1992), *Delivery Man Problem* (Fischetti et al., 1993), *Cumulative Traveling Salesman Problem* (Bianco et al., 1993) and *School Bus Driver Problem* (Chaudhuri et al., 2003). The constraint on the tour origin at 0 is sometimes not considered in early articles, but this variant can be tackled in our context by adding a dummy depot such that $t_{0j} = t_{j0} = 0$ for $j \in V$.

The MLP was proven $\mathcal{NP}$-hard for general metric spaces (Sahni & Gonzalez, 1976) and also when the subjacent structure is an edge-weighted tree (Sitters, 2002). For structures such as paths, unweighted trees and trees with diameter at most 3, polynomial-time algorithms based on dynamic programming have been proposed (Blum, Chalasanit, Pulleyblankt, Raghavan, & Sudan, 1994; García, Jodrá, & Tejel, 2002; Wu, Huang, & Zhan, 2004). Although the MLP seems to be a simple TSP variant, one can verify that the former has properties that are not present in the latter. One of them is that small local modifications in the configuration of the input points can lead to highly nonlocal changes in the structure of an optimal solution (Blum et al., 1994; Goemans & Kleinberg, 1998). Another feature of the MLP is the nonlocal character of the objective function, as an additional arc inserted in the beginning of the circuit affects the latency of all remaining vertices (Arora & Karakostas, 2003).

Real-life applications of the MLP often arise from distribution systems, where some quality criterion regarding the customer satisfaction must be focused. The MLP considers waiting times (latency) of a service system from the customer's point of view, i.e., while in the MLP the objective is to minimize the average waiting time of each customer, in the TSP the objective is to minimize the total time required to visit all customers. In view of this, one can say that the MLP is customer oriented, while the TSP is server oriented (Archer & Williamson, 2003). Therefore, the MLP can be employed in the modeling of different types of service systems. Important practical applications can be found in home delivery services (Méndez-Dıaz, Zabala, & Lucena, 2008), logistics for emergency relief (Campbell, Vandenbussche, & Hermann, 2008) and data retrieval in computer networks (Ezzine, Semet, & Chabchoub, 2010). However, although the MLP appears in several important settings, this problem has not received sufficient attention in the literature so far. In particular, few efficient heuristics have been designed to tackle problems of realistic size. Moreover, current exact methods are not capable of consistently solving instances with more than 100 customers.

The contributions of this work are twofold. The first one is to present a simple and effective metaheuristic for the MLP, which combines components of Greedy Randomized Adaptive Search Procedure (GRASP) (Feo and Resende, 1995), Iterated Local Search (ILS)

* Corresponding author. Tel.: +55 21 2629 5665; fax: +55 21 2629 5666.
E-mail addresses: mmsilva@ic.uff.br (M.M. Silva), anand@ct.ufpb.br (A. Subramanian), thibaut.vidal@cirrelt.ca (T. Vidal), satoru@ic.uff.br (L.S. Ochi).

0377-2217/$ - see front matter © 2012 Elsevier B.V. All rights reserved.
http://dx.doi.org/10.1016/j.ejor.2012.03.044

(Lourenço et al., 2003) and Variable Neighborhood Descent with Random neighborhood ordering (RVND) (Mladenović and Hansen, 1997; Subramanian et al., 2010). The second contribution is a simple move evaluation procedure that requires $\mathcal{O}(1)$ amortized elementary operations. Such procedure can be applied to any neighborhood structure based on a bounded number of arc exchanges and thus to all classical neighborhoods used in the MLP literature. The proposed solution approach is easy to implement and relies on very few parameters. Extensive computational experiments on benchmark instances involving up to 1000 customers underline the remarkable performance of this method, both in terms of solution quality and computational efficiency. Known optimal solutions for problems with up to 107 customers are also systematically obtained in a few seconds.

The remainder of the paper is organized as follows. Section 2 presents some related works. Section 3 describes the proposed algorithm. Section 4 contains the computational results. Finally, Section 5 presents the concluding remarks of this work.

2. Related works

Several exact and non-exact approaches were proposed to solve the MLP in the literature. However, as shown in the following, exact algorithms are still limited to small problem sizes, while few efficient heuristic procedures have been designed.

Lucena (1990) proposed an early exact enumerative algorithm, relying on a non-linear integer formulation in which lower bounds are derived using a Lagrangian relaxation. Bianco et al. (1993) put forward two exact algorithms that incorporate lower bounds obtained via a Lagrangian relaxation. Fischetti et al. (1993) proposed an enumerative algorithm that makes use of lower bounds obtained from a linear integer programming formulation. Van Eijl (1995) and Méndez-Díaz et al. (2008) developed Mixed Integer Programming (MIP) formulations. The latter also introduced various families of valid inequalities which were evaluated using a branch-and-cut algorithm. Ezzine et al. (2010) proposed two new integer programming formulations and their linear relaxations were evaluated by means of computational experiments. Bigras, Gamache, and Savard (2008) suggested a number of integer programming formulations as well as a branch-and-bound algorithm. Abeledo, Fukasawa, Pessoa, and Uchoa (2010a, 2010b) developed a branch-cut-and-price approach over an extended formulation as well as several families of facet defining inequalities. To our knowledge, their solution method was the only one capable of solving instances with up to 107 customers.

Some approximation algorithms are also known for the MLP (Blum et al., 1994; Goemans & Kleinberg, 1998; Arora & Karakostas, 2003; Ausiello, Leonardi, & Marchetti-Spaccamela, 2000; Archer & Williamson, 2003; Chaudhuri et al., 2003; Fakcharoenphol, Harrelson, & Rao, 2007; Nagarajan & Ravi, 2008; Archer & Blasiak, 2010). The first one was suggested by Blum et al. (1994) with an approximation factor of 144. For general metric spaces, the smallest approximation factor of 3.59 was found by Chaudhuri et al. (2003). For the case where an edge-weighted tree is considered, the smallest approximation factor, of 3.03, was obtained by Archer and Blasiak (2010).

Up to this date, few metaheuristics are available for the MLP. Salehipour, Sörensen, Goos, and Bräysy (2011) put forward a heuristic algorithm based on GRASP, Variable Neighborhood Descent (VND) and Variable Neighborhood Search (VNS) with a shaking procedure based on random relocations and neighborhood restrictions to spatially close customers. A Tabu Search (TS) was also developed by Dewilde, Cattrysse, Coene, Spieksma, and Vansteenwegen (2010) for the MLP with profits. Other approaches were designed primarily for the cumulative vehicle routing problem (CCVRP), but can also tackle the MLP as a special case with a single vehicle. The memetic algorithm of Ngueveu, Prins, and Wolfler Calvo (2010), especially, introduces new move evaluation procedures in $\mathcal{O}(1)$ operations for some particular neighborhood structures and produces high quality solutions on the previously mentioned MLP benchmark. Finally, Ribeiro and Laporte (2012) developed an adaptive large neighborhood search, which performed well in solving the CCVRP, but it was not tested on the MLP.

It is worth noting that the MLP and the TSP appear as special cases of a more general problem known as the Time-Dependent TSP (TDTSP). In this problem, the cost associated with the travel path between two vertices depends not only on their localization in the metric space but also on the position in which they appear in the circuit. The objective is to minimize the total cost of visiting all nodes (Abeledo et al., 2010a, 2010b; Blum et al., 1994; Lucena, 1990). The TDTSP and the MLP can be also seen as single-machine scheduling problems with sequence-dependent processing times (Bigras et al., 2008; Gouveia & Voss, 1995; Picard & Queyranne, 1978) and under a "flow-time" performance measure (Tsitsiklis, 1992). Finally, other MLP variants can be also found in the literature. The variant with time windows was studied by Heilporn, Cordeau, and Laporte (2010), Tsitsiklis (1992) and Van Eijl (1995). The case with asymmetric costs was examined by Nagarajan and Ravi (2008) and the variant with multiple servers was tackled by Fakcharoenphol et al. (2007).

3. Proposed algorithm

The simple and efficient metaheuristic proposed here and called GILS-RVND brings together the components of GRASP, ILS and RVND. The pseudocode of the developed approach is presented in Algorithm 1. The method performs I_{Max} iterations (lines 3–21), where in each of which an initial solution is generated using a greedy randomized procedure. The level of greediness is controlled by a parameter α, which is randomly chosen among the values of a given set R. Each initial solution is then improved by means of a RVND procedure combined with a perturbation mechanism in the spirit of ILS (lines 8–15), which is run until I_{ILS} consecutive perturbations without improvements are performed. It is important to mention that the perturbation is always performed on the best current solution s' of a given iteration (acceptance criterion). Finally, the heuristic returns the best solution s^* among all iterations.

Algorithm 1. GILS-RVND

```
1  Procedure GILS-RVND (I_Max, I_ILS, R)
2  f* ← ∞;
3  for i ← 1, ..., I_Max do
4     α ← random value ∈R;
5     s ← Construction (α);
6     s' ← s;
7     iterILS ← 0;
8     while iterILS < I_ILS do
9        s ← RVND (s);
10       if f(s) < f(s') then
11          s' ← s;
12          iterILS ← 0;
13       endif
14       s ← Perturb (s');
15       iterILS ← iterILS + 1;
16    endwhile
17    if f(s') < f* then
18       s* ← s';
19       f* ← f(s');
20    endif
21 endfor
22 return s*;
23 end GILS-RVND
```

3.1. Constructive procedure

The constructive procedure, used to generate initial solutions, is described in Algorithm 2. Firstly, a partial solution s is initialized with a vertex associated to the depot (line 2), whereas a Candidate List (CL) is initialized with the remaining vertices (lines 3 and 4). In the main loop (lines 6–13), CL is sorted in ascending order according to the nearest neighbor criterion with respect to the last vertex added to s (line 7). A Restricted Candidate List (RCL) (line 8) is then built by considering only the α% best candidates of CL. Next, a customer is chosen at random from RCL and added to s (lines 9 and 10). When the set of the α% best candidates is of size less than one or when $\alpha = 0$, the algorithm chooses the best candidate. The constructive procedure terminates when all customers are added to s.

Algorithm 2. Construction

```
1  Procedure Construction (α)
2  s ∪ {0};
3  Initialize Candidate List CL;
4  CL ← CL − {0};
5  r ← 0;
6  while CL ≠ ∅ do
7     Sort CL in ascending order according to their distance with
      respect to r;
8     Update RCL considering only the α% best candidates of CL;
9     Choose c ∈ RCL at random;
10    s ∪ {c};
11    r ← c;
12    CL ← CL − {r};
13 endwhile
14 return s;
15 end Construction
```

3.2. Improvement procedure with efficient move evaluations

The local search is performed by a method based on RVND. Let t be the number of neighborhood structures and $N = \{N^1, N^2, N^3, \ldots, N^t\}$ be their corresponding set. Whenever a given neighborhood of the set N fails to improve the current best solution, RVND randomly selects another neighborhood from the same set to continue the search. Preliminary tests revealed that this approach is capable of finding better solutions as compared to those that adopt a deterministic order.

The set N is composed of the following five well-known TSP neighborhood structures, whose associated solutions are explored in an exhaustive fashion with a best improvement strategy.

- **Swap**—$N^{(1)}$—Two customers of the tour are interchanged.
- **2**-opt—$N^{(2)}$—Two non-adjacent arcs are removed and another two are inserted in order to build a new feasible tour.
- **Reinsertion**—$N^{(3)}$—One customer is relocated to another position of the tour.
- **Or**-opt2—$N^{(4)}$—Two adjacent customers are reallocated to another position of the tour.
- **Or**-opt3—$N^{(5)}$—Three adjacent customers are reallocated to another position of the tour.

Algorithm 3. RVND

```
1  Procedure RVND (s)
2  Initialize the Neighborhood List NL;
3  Initialize re-optimization data structures on subsequences;
4  while NL ≠ ∅ do
5     Choose a neighborhood N^(η) ∈ NL at random;
6     Find the best neighbor s' of s ∈ N^(η);
7     if f(s') < f(s) then
8        s ← s';
9        f(s) ← f(s');
10       Update NL;
11       Update re-optimization data structures;
12    else
13       Remove N^(η) from the NL;
14    endif
15 endwhile
16 return s;
17 end RVND
```

It is worth emphasizing that neighbor evaluations in the MLP case are less straightforward than for the classic TSP, since the time of service of a large proportion of customers are impacted during moves. A direct cost evaluation procedure involves examining customer visits in the sequence order with a view of computing their latencies and thus the total cost. This method unfortunately leads to $\mathcal{O}(n)$ operations for each move evaluation, resulting in $\mathcal{O}(n^3)$ operations for a full neighborhood search. In the spirit of the feasibility checking approach of Savelsbergh (1985), originally designed for the vehicle routing problem with time windows, Ngueveu et al. (2010) proposed a move evaluation procedure, which requires $\mathcal{O}(1)$ amortized operations. Such procedure is based on the management of global information on partial routes.

We follow this line of thought and propose a very simple move evaluation approach, also requiring $\mathcal{O}(1)$ amortized operations, which can be generally applied to any neighborhood structure based on a bounded number of arc exchanges. The approach relies on a re-optimization framework "by concatenation", originally developed by Kindervater and Savelsbergh (1997) and extended by Vidal, Crainic, Gendreau, and Prins (2011) to a wide range of move evaluation settings presenting temporal characteristics. Indeed, any arc exchange-based move involves separating the visit sequence into several subsequences, which are then concatenated together. We thus introduce some "re-optimization data structures" to characterize the cost of subsequences $\sigma = (\sigma_u, \ldots, \sigma_v)$, and show how to update them on larger subsequences by induction on the concatenation operator, here defined as $\oplus$. The following data structures are used:

- Duration $T(\sigma)$ to perform the visits sequence σ.
- Cost $C(\sigma)$ to perform the sequence, when starting at time 0.
- Delay cost $W(\sigma)$, related to a one time unit delay in the starting time.

For a sequence with a single vertex $\sigma^0 = i$, the duration $T(\sigma^0)$ is 0 by default since there is no travel time. The constant cost $C(\sigma^0)$ is set to 0, whereas the delay cost $W(\sigma^0)$ is set to 1 when the vertex is a customer, otherwise $W(\sigma^0) = 0$. Proposition 1 then enables to compute these values on larger subsequences by induction on the concatenation operator.

Proposition 1. *Let* $\sigma = (\sigma_u, \ldots, \sigma_v)$ *and* $\sigma' = (\sigma'_w, \ldots, \sigma'_x)$ *be two sub-sequences of visits. The sub-sequence* $\sigma \oplus \sigma' = (\sigma_u, \ldots, \sigma_v, \sigma'_w, \ldots, \sigma'_x)$ *is characterized by the following values:*

$$T(\sigma \oplus \sigma') = T(\sigma) + t_{\sigma_v \sigma'_w} + T(\sigma') \quad (1)$$

$$C(\sigma \oplus \sigma') = C(\sigma) + W(\sigma')(T(\sigma) + t_{\sigma_v \sigma'_w}) + C(\sigma') \quad (2)$$

$$W(\sigma \oplus \sigma') = W(\sigma) + W(\sigma') \quad (3)$$

The proposed neighborhood evaluation procedure relies on Proposition 1 to compute the cost of relevant sub-sequences (and their reversal for 2-opt moves), during a preprocessing phase in $\mathcal{O}(n^2)$ operations; and then to evaluate the cost $C(\sigma)$ of tours issued from moves as the concatenation of several sub-sequences. Classical neighborhoods in the literature correspond to a concatenation of less than five sub-sequences. Hence, when data on sub-sequences has been processed, any move evaluation is performed

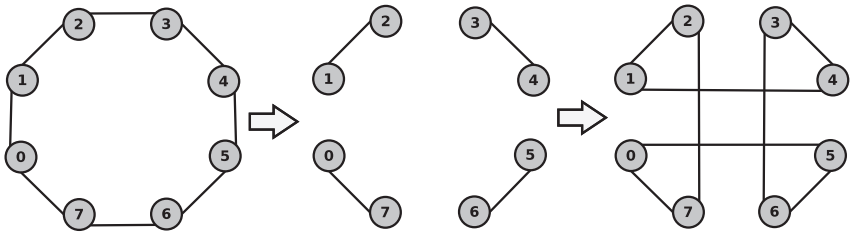


Fig. 1. Double-bridge.

Table 1
Results for TSPLIB instances selected by Abeledo et al. (2010a, 2010b).

Instance	Abeledo et al.	GILS-RVND			
	OPT or UB	Best Sol.	Avg. Sol.	Avg. Gap (%)	Avg. Time (s)
dantzig42	12,528	12,528	12528.00	0.00	0.16
swiss42	22,327	22,327	22327.00	0.00	0.16
att48	209,320	209,320	209320.00	0.00	0.32
gr48	102,378	102,378	102378.00	0.00	0.33
hk48	247,926	247,926	247926.00	0.00	0.30
eil51	10,178	10,178	10178.00	0.00	0.49
berlin52	143,721	143,721	143721.00	0.00	0.46
brazil58	512,361	512,361	512361.00	0.00	0.78
st70	20,557	20,557	20557.00	0.00	1.65
eil76	17,976	17,976	17976.00	0.00	2.64
pr76	3,455,242	3,455,242	3455242.00	0.00	2.31
pr76r	345,427	345,427	345427.00	0.00	2.41
gr96	2,097,170	2,097,170	2097170.00	0.00	6.19
rat99	58,288[a]	**57,986**	**57986.00**	−0.52	11.27
kroA100	983,128	983,128	983128.00	0.00	8.59
kroB100	986,008	986,008	986008.00	0.00	9.21
kroC100	961,324	961,324	961324.00	0.00	8.17
kroD100	976,965	976,965	976965.00	0.00	8.46
kroE100	971,266	971,266	971266.00	0.00	8.31
rd100	340,047	340,047	340047.00	0.00	8.52
eil101	27,519[a]	**27,513**	**27513.00**	−0.02	12.76
lin105	603,910	603,910	603910.00	0.00	8.42
pr107	2,026,626	2,026,626	2026626.00	0.00	10.89

[a] Optimality is not proven for this instance.

Table 2
Results for TSPLIB instances selected by Salehipour et al. (2011).

Instance	Salehipour et al.	GILS-RVND			
	Best Sol.	Best Sol.	Avg. Sol.	Avg. Gap (%)	Avg. Time (s)
st70	19,553	**19,215**	**19215.00**	−1.73	1.51
rat99	56,994	**54,984**	**54984.00**	−3.53	9.47
kroD100	976,830	**949,594**	**949594.00**	−2.79	6.90
lin105	585,823	585,823	585823.00	0.00	6.19
pr107	1,983,475	**1,980,767**	**1980767.00**	−0.14	8.13
rat195	213,371	**210,191**	**210335.90**	−1.42	75.56
pr226	7,226,554	**7,100,308**	**7100308.00**	−1.75	59.05
lin318	5,876,537	**5,560,679**	**5569819.50**	−5.22	220.59
pr439	18,567,170	**17,688,561**	**17734922.00**	−4.48	553.74
att532	18,448,435	5,581,240	5597866.80	–	1792.61
Average				−2.34	

in a constant number of operations and thus a full neighborhood is performed in $\mathcal{O}(n^2)$ operations. An illustrative example of the re-optimization data structures and the cost evaluation is given in Appendix A.

The pseudocode of the RVND procedure is described in Algorithm 3. Firstly, the Neighborhood List (NL) is initialized with all neighborhood structures, as well as the re-optimization data structures (lines 2 and 3). In the main loop (lines 4–15), a given neighborhood structure $N^{(\eta)}$ is selected at random from the NL (line 5). Neighbor solutions are evaluated in $\mathcal{O}(1)$ operations, using the re-optimization data structures and the best found solution is stored in s' (line 6). In case of improvement, NL is repopulated with all neighborhood structures and the re-optimization data structures are updated (lines 7–12). Otherwise, $N^{(\eta)}$ is removed from NL (line 13). The procedure terminates when NL becomes empty.

3.3. Perturbation mechanism

When the local search fails to improve a solution s, a perturbation is applied over the best current solution s' of the corresponding GILS-RVND iteration. The perturbation mechanism, called double-bridge, was originally developed by Martin, Otto, and Felten (1991) for the TSP. It consists in removing and re-inserting four arcs in such a way that a feasible tour is generated (see Fig. 1). This mechanism can also be seen as a permutation of two disjoint segments of a tour.

4. Computational results

The algorithm was coded in C++ (g++ 4.4.3) and executed on an Intel® Core™ i7 2.93 GHz, with 8.0 GB of RAM memory running under GNU/Linux Ubuntu 10.04 (*kernel* 2.6.32–25). Only a single thread was used in the experiments.

Through preliminary tests, we observed that the values $I_{Max} = 10$, $I_{ILS} = \min\{100, n\}$ and $R = \{0.00, 0.01, 0.02, \ldots, 0.25\}$ resulted in a good trade-off between solution quality and run time. This parameter setting has thus been used in the following experiments.

GILS-RVND was tested on 9 sets of benchmark instances. Seven of these sets were generated by Salehipour et al. (2011), where each of them is composed of 20 instances with 10, 20, 50, 100, 200, 500 and 1000 customers, respectively. Another set of 10 instances ranging from 70 to 532 customers was selected from the TSPLIB (Reinelt (1991)) by the same authors. It is important to mention that Salehipour et al. (2011) considered the version where the objective is to find the minimum latency over a Hamiltonian path starting from vertex 0. Finally, we present results on a last subset of instances from the TSPLIB, selected by Abeledo et al. (2010a, 2010b) and composed of 23 test-problems ranging between 42 and 107 customers.

Until this date, optimal solutions were only reported by Salehipour et al., 2011 for instances with up to 20 customers. We had access to the source code of the BCP approach of Abeledo et al., 2010a, and ran their algorithm to find the optimal solutions of all instances with up to 50 customers. Salehipour et al. (2011)

Table 3
Results obtained on the small instances generated by Salehipour et al. (2011), involving 10, 20 and 50 customers.

Instance	OPT		
	S10	S20	S50
TRP-S*n*-R1	1303	3175	12,198
TRP-S*n*-R2	1517	3248	11,621
TRP-S*n*-R3	1233	3570	12,139
TRP-S*n*-R4	1386	2983	13,071
TRP-S*n*-R5	978	3248	12,126
TRP-S*n*-R6	1477	3328	12,684
TRP-S*n*-R7	1163	2809	11,176
TRP-S*n*-R8	1234	3461	12,910
TRP-S*n*-R9	1402	3475	13,149
TRP-S*n*-R10	1388	3359	12,892
TRP-S*n*-R11	1405	2916	12,103
TRP-S*n*-R12	1150	3314	10,633
TRP-S*n*-R13	1531	3412	12,115
TRP-S*n*-R14	1219	3297	13,117
TRP-S*n*-R15	1087	2862	11,986
TRP-S*n*-R16	1264	3433	12,138
TRP-S*n*-R17	1058	2913	12,176
TRP-S*n*-R18	1083	3124	13,357
TRP-S*n*-R19	1394	3299	11,430
TRP-S*n*-R20	951	2796	11,935

Table 4
Results on the 100-customers instances generated by Salehipour et al. (2011).

Instance	UB	GILS-RVND			
		Best Sol.	Avg. Sol.	Avg. Gap (%)	Avg. Time (s)
TRP-S100-R1	35,334	32,779	32779.00	−7.23	7.05
TRP-S100-R2	38,442	33,435	33435.00	−13.02	7.51
TRP-S100-R3	37,642	32,390	32390.00	−13.95	7.07
TRP-S100-R4	37,508	34,733	34733.00	−7.40	7.27
TRP-S100-R5	37,215	32,598	32598.00	−12.41	8.87
TRP-S100-R6	40,422	34,159	34159.00	−15.49	7.82
TRP-S100-R7	37,367	33,375	33375.00	−10.68	8.74
TRP-S100-R8	38,086	31,780	31780.00	−16.56	7.08
TRP-S100-R9	36,000	34,167	34167.00	−5.09	7.47
TRP-S100-R10	37,761	31,605	31605.00	−16.30	6.78
TRP-S100-R11	37,220	34,188	34188.00	−8.15	7.75
TRP-S100-R12	34,785	32,146	32146.00	−7.59	7.20
TRP-S100-R13	37,863	32,604	32604.00	−13.89	7.78
TRP-S100-R14	36,362	32,433	32433.00	−10.81	6.29
TRP-S100-R15	39,381	32,574	32574.00	−17.28	7.13
TRP-S100-R16	39,823	33,566	33566.00	−15.71	7.97
TRP-S100-R17	41,824	34,198	34198.00	−18.23	9.23
TRP-S100-R18	39,091	31,929	31929.00	−18.32	7.07
TRP-S100-R19	39,941	33,463	33463.00	−16.22	8.08
TRP-S100-R20	39,888	33,632	33632.00	−15.68	8.73
Average				−13.00	

Table 5
Results on the 200-customers instances generated by Salehipour et al. (2011).

Instance	UB	GILS-RVND			
		Best Sol.	Avg. Sol.	Avg. Gap (%)	Avg. Time (s)
TRP-S200-R1	105,044	88,787	88794.60	−15.47	73.39
TRP-S200-R2	104,073	91,977	92013.10	−11.59	68.07
TRP-S200-R3	111,644	92,568	92631.20	−17.03	67.11
TRP-S200-R4	104,956	93,174	93192.30	−11.21	72.17
TRP-S200-R5	101,912	88,737	88841.20	−12.83	70.77
TRP-S200-R6	103,751	91,589	91601.90	−11.71	70.52
TRP-S200-R7	109,810	92,754	92763.20	−15.52	72.80
TRP-S200-R8	103,830	89,048	89049.00	−14.24	75.15
TRP-S200-R9	100,946	86,326	86326.00	−14.48	65.45
TRP-S200-R10	108,061	91,552	91596.50	−15.24	74.25
TRP-S200-R11	103,297	92,655	92700.60	−10.26	73.15
TRP-S200-R12	107,715	91,457	91504.10	−15.05	76.74
TRP-S200-R13	100,505	86,155	86181.40	−14.25	72.96
TRP-S200-R14	107,543	91,882	91929.10	−14.52	70.94
TRP-S200-R15	100,196	88,912	88912.40	−11.26	70.41
TRP-S200-R16	104,462	89,311	89364.70	−14.45	77.89
TRP-S200-R17	107,216	89,089	89118.30	−16.88	71.17
TRP-S200-R18	108,148	93,619	93676.60	−13.38	77.03
TRP-S200-R19	105,716	93,369	93401.60	−11.65	71.08
TRP-S200-R20	116,676	86,292	86292.00	−26.04	70.61
Average				−14.35	

and Ngueveu et al. (2010) also did not report detailed results for every instance, but only the average gap between their solutions and those obtained by a greedy nearest neighbor heuristic on each group of instances. Hence, in order to compare our heuristic with the best methods developed by Salehipour et al. (2011) and Ngueveu et al. (2010), we report the average solution quality of GILS-RVND over 10 runs, for each instance, relative to the nearest neighbor heuristic, which is equivalent to our constructive procedure with $\alpha = 0$.

In the tables presented hereafter, **Instance** denotes the instance, **OPT** is the optimal solution, **UB** indicates the upper bound obtained by the nearest neighbor heuristic, **Best Sol.** corresponds to the best solution obtained by GILS-RVND and **Avg. Sol** denotes the average solution of 10 executions found by GILS-RVND. **Avg. Time** is the average time in seconds of 10 executions of the proposed algorithm, while **cTime** represents scaled run times, estimated on a Pentium IV by means of the factors of Dongarra (2011). Finally, **Avg. Gap** is the average gap between the average solution and either the optimal solution (Avg. Gap = (100(Avg. Sol − OPT)/OPT)), when available, or the UB given by the nearest neighbor heuristic (Avg. Gap = (100(Avg. Sol − UB)/UB)). Improved solutions are highlighted in boldface.

Table 1 compares the solutions obtained by the exact algorithm of Abeledo et al. (2010a) and those found by GILS-RVND for the set of 23 test-problems selected from the TSPLIB by Abeledo et al. (2010a). GILS-RVND found the optimal solutions, when available, in all executions. Moreover, new best solutions were found for the two remaining open instances (rat99 and eil101).

Table 2 compares the best solutions obtained by Salehipour et al. (2011) and those found by GILS-RVND for the set of 10 instances selected from the TSPLIB by Salehipour et al. (2011). It can be observed that GILS-RVND was capable of equaling or

Table 6
Results on the 500-customers instances generated by Salehipour et al. (2011).

Instance	UB	GILS-RVND			
		Best Sol.	Avg. Sol.	Avg. Gap (%)	Avg. Time (s)
TRP-S500-R1	2,192,688	1,841,386	1856018.70	−15.35	1738.48
TRP-S500-R2	2,176,449	1,816,568	1823196.90	−16.23	1476.13
TRP-S500-R3	2,261,125	1,833,044	1839254.20	−18.66	1557.48
TRP-S500-R4	2,088,773	1,809,266	1815876.40	−13.06	1597.06
TRP-S500-R5	2,216,937	1,823,975	1834031.70	−17.27	1530.94
TRP-S500-R6	2,137,187	1,786,620	1790912.40	−16.20	1576.91
TRP-S500-R7	2,212,936	1,847,999	1857926.60	−16.04	1584.67
TRP-S500-R8	2,132,165	1,820,846	1829257.30	−14.21	1565.01
TRP-S500-R9	2,141,458	1,733,819	1737024.90	−18.89	1409.23
TRP-S500-R10	2,163,387	1,762,741	1767366.30	−18.31	1621.85
TRP-S500-R11	2,288,538	1,797,881	1801467.90	−21.28	1530.98
TRP-S500-R12	2,081,530	1,774,452	1783847.10	−14.30	1554.75
TRP-S500-R13	2,080,370	1,873,699	1878049.40	−9.73	1598.46
TRP-S500-R14	2,051,683	1,799,171	1805732.90	−11.99	1701.90
TRP-S500-R15	2,035,804	1,791,145	1797532.90	−11.70	1623.79
TRP-S500-R16	2,142,426	1,810,188	1816484.00	−15.21	1583.70
TRP-S500-R17	2,117,999	1,825,748	1834443.20	−13.39	1549.80
TRP-S500-R18	2,159,400	1,826,263	1833323.70	−15.10	1620.02
TRP-S500-R19	2,009,335	1,779,248	1782763.90	−11.28	1602.87
TRP-S500-R20	2,155,026	1,820,813	1830483.30	−15.06	1507.96
Average				−15.16	

Table 7
Results on the 1000-customers instances generated by Salehipour et al. (2011).

Instance	UB	GILS-RVND			
		Best Sol.	Avg. Sol.	Avg. Gap (%)	Avg. Time (s)
TRP-S1000-R1	6,030,081	5,107,395	5133698.30	−14.87	31894.51
TRP-S1000-R2	6,282,704	5,106,161	5127449.40	−18.39	30881.19
TRP-S1000-R3	5,874,496	5,096,977	5113302.90	−12.96	30184.15
TRP-S1000-R4	5,892,443	5,118,006	5141392.60	−12.75	29951.12
TRP-S1000-R5	6,192,250	5,103,894	5122660.70	−17.27	30129.51
TRP-S1000-R6	6,056,173	5,115,816	5143087.10	−15.08	28161.57
TRP-S1000-R7	5,973,701	5,021,383	5032722.00	−15.75	25945.41
TRP-S1000-R8	6,046,965	5,109,325	5132722.60	−15.12	26572.71
TRP-S1000-R9	6,159,862	5,052,599	5073245.30	−17.64	26330.40
TRP-S1000-R10	5,843,354	5,078,191	5093592.60	−12.83	25676.31
TRP-S1000-R11	6,057,225	5,041,913	5066161.50	−16.36	26235.63
TRP-S1000-R12	5,996,323	5,029,792	5051235.20	−15.76	27910.11
TRP-S1000-R13	6,052,162	5,102,520	5131437.50	−15.21	28475.89
TRP-S1000-R14	5,952,120	5,099,433	5118980.60	−14.00	27639.81
TRP-S1000-R15	5,934,175	5,142,470	5174493.20	−12.80	27633.07
TRP-S1000-R16	5,925,180	5,073,972	5090280.50	−14.09	26653.16
TRP-S1000-R17	6,068,380	5,071,485	5084450.40	−16.21	27503.43
TRP-S1000-R18	6,169,728	5,017,589	5037094.00	−18.36	28808.09
TRP-S1000-R19	6,004,893	5,076,800	5097167.60	−15.12	29637.49
TRP-S1000-R20	6,159,355	4,977,262	5002920.60	−18.78	27499.24
Average				−15.47	

Table 8
Comparison among different solution approaches.

n	Salehipour et al.		Ngueveu et al.		GILS-RVND		
	UB (%)	Time (s)	UB (%)	cTime (s)	UB (%)	Avg. Time (s)	cTime (s)
10	−2.44	0.00	−2.43	0.00	−2.44	0.00	0.00
20	−9.86	0.04	−10.11	0.01	−10.28	0.02	0.05
50	−9.74	3.54	−9.36	1.44	−11.01	0.55	1.36
100	−11.66	103.92	−11.95	93.26	−13.00	7.64	18.94
200	−16.21	3995.00	−12.81	938.16	−14.35	72.08	178.72
500	−9.71	10381.36	−13.85	16208.70	−15.16	1576.60	3909.03
1000	–	–	–	–	−15.47	28186.14	69884.87
Average	−9.94	2413.06	−10.09	2873.60	−11.04	276.15	684.68
					−11.67[a]	4263.29[a]	10570.42[a]

[a] Average considering the instances with 1000 customers.

improving the best known solutions in all cases. The average gap between the average solutions obtained by GILS-RVND and best solutions reported by Salehipour et al. (2011) was −2.34%. When computing the average, we did not consider the result of the last instance, called "att532", since the large gap between the two methods seemed abnormal.

Table 3 presents the average results of GILS-RVND on the instances of size 10, 20 and 50, generated by Salehipour et al. (2011). These solutions are compared with optimal values, found either by Salehipour et al. (2011) for problems with less than 20 customers, or by Abeledo et al. (2010a) for problems with 50 customers. It is noteworthy that GILS-RVND found optimal solutions during all executions, with an average computational time of 0.01, 0.02 and 0.55 seconds, respectively.

Tables 4–7 compare the results of GILS-RVND with the upper bounds obtained using the nearest neighbor heuristic on the remaining instances suggested by Salehipour et al. (2011) with 100, 200, 500 and 1000 customers. As expected, the proposed method largely improves upon the nearest neighbor heuristic, with average gaps of −13.00%, −14.35%, −15.16% and −15.47%, respectively.

Finally, Table 8 presents a summarized comparison between the average gaps, with respect to the nearest neighbor heuristic, found by GILS-RVND, Salehipour et al. (2011) (GRASP + VNS deep version) and Ngueveu et al. (2010) (memetic deep version) for the seven sets of instances of Salehipour et al. (2011). The proposed algorithm led to larger improvements for most instance categories and also required smaller scaled computational time, thus outperforming previous methods.

5. Concluding remarks

This paper introduced a new hybrid metaheuristic for the Minimum Latency Problem, which gathers several successful concepts from GRASP, ILS, RVND, along with simple move evaluation procedures in $\mathcal{O}(1)$ time. The latter methodology can be applied to any neighborhood structure based on a bounded number of arc exchanges or visit relocations. The overall approach is simple to describe and to implement. Its effectiveness, in terms of both solution quality and computational time, was assessed by extensive experiments on 173 benchmark instances containing up to 1000 customers. The method systematically obtains the optimal solution in a few seconds on instances with up to 107 customers, where this value is known. Moreover, all best known solutions of the benchmark instances have been either equaled or improved.

Promising avenues of research involve extending the proposed algorithm to other MLP variants such as the MLP with profits and the version with multiple vehicles.

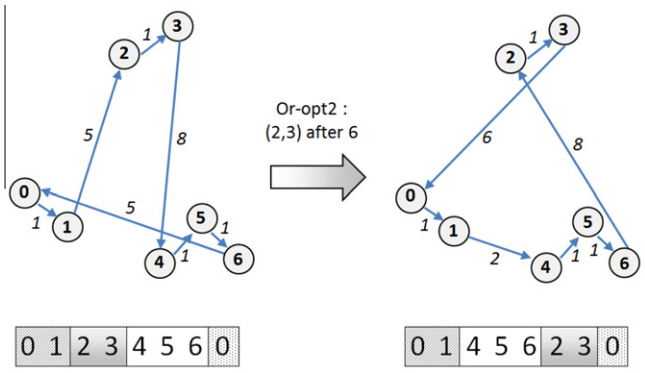


Fig. A.2. Or-opt2 move as a concatenation of four sub-sequences.

Table A.9
Preprocessed data structures and partial results for a sequence evaluation.

σ	$T(\sigma)$	$C(\sigma)$	$W(\sigma)$
(0, 1)	1	1	1
(4, 5, 6)	2	3	3
(2, 3)	1	1	2
(0)	0	0	0
(0, 1) ⊕ (4, 5, 6)	5	13	4
(0, 1) ⊕ (4, 5, 6) ⊕ (2, 3)	14	40	6
(0, 1) ⊕ (4, 5, 6) ⊕ (2, 3) ⊕ (0)	20	**40**	6

Acknowledgments

We would like to thank Prof. Eduardo Uchoa and Prof. Artur Pessoa for providing access to their BCP code. This research was partially supported by the following Brazilian research agencies: CNPq, CAPES and FAPERJ.

Appendix A. Example of efficient move evaluation for the MLP

Fig. A.2 presents a numerical example on a small problem with five vertices. The travel time of each arc is indicated in italics. An Or-opt2 move is illustrated, involving the relocation of customers 2 and 3 after customer 6. Starting from the sequence (0, 1, 2, 3, 4, 5, 6, 0), this move produces a concatenation of four sub-sequences (0, 1) ⊕ (4, 5, 6) ⊕ (2, 3) ⊕ (0).

The relevant preprocessed data structures are reported in the top of Table A.9. The cost of the new sequence is then obtained by applying the concatenations equations three times, to compute $C((0, 1) \oplus (4, 5, 6))$ and so on.

References

Abeledo, H., Fukasawa, R., Pessoa, A., Uchoa, E., 2010a. The Time Dependent Traveling Salesman Problem: Polyhedra and Algorithm. Tech. Rep. RPEP, vol. 10 no. 15. Universidade Federal Fluminense, Brasil.

Abeledo, H.G., Fukasawa, R., Pessoa, A.A., Uchoa, E., 2010b. The time dependent traveling salesman problem: polyhedra and branch-cut-and-price algorithm. In: Proceedings of the 9th International Symposium on Experimental Algorithms, SEA 2010, pp. 202–213.

Archer, A., Blasiak, A., 2010. Improved approximation algorithms for the minimum latency problem via prize-collecting strolls. In: Proceedings of the 21th Annual ACM-SIAM Symposium on Discrete Algorithms. pp. 429–447.

Archer, A., Williamson, D.P., 2003. Faster approximation algorithms for the minimum latency problem. In: Proceedings of the 40th Annual ACM-SIAM Symposium on Discrete Algorithms, pp. 88–96.

Arora, S., Karakostas, G., 2003. Approximation schemes for minimum latency problems. SIAM Journal on Computing 32 (5), 1317–1337.

Ausiello, G., Leonardi, S., Marchetti-Spaccamela, A., 2000. On salesmen, repairmen, spiders, and other traveling agents. In: Proceedings of the 4th Italian Conference on Algorithms and Complexity, pp. 1–16.

Bianco, L., Mingozzi, A., Ricciardelli, S., 1993. The traveling salesman problem with cumulative costs. Networks 23 (2), 81–91.

Bigras, L.-P., Gamache, M., Savard, G., 2008. The time-dependent traveling salesman problem and single machine scheduling problems with sequence dependent setup times. Discrete Optimization 5 (4), 685–699.

Blum, A., Chalasanit, P., Pulleyblankt, B., Raghavan, P., Sudan, M., 1994. The minimum latency problem. In: Proceedings of the 26th Annual ACM Symposium on Theory of Computing, pp. 163–171.

Campbell, A., Vandenbussche, D., Hermann, W., 2008. Routing for relief efforts. Transportation Science 42 (2), 127–145.

Chaudhuri, K., Godfrey, B., Rao, S., Talwar, K., 2003. Paths, trees, and minimum latency tours. In: Proceedings of the 44th Annual IEEE Symposium on Foundations of Computer Science, FOCS 2003, pp. 36–45.

Dewilde, T., Cattrysse, D., Coene, S., Spieksma, F.C.R., Vansteenwegen, P., 2010. Heuristics for the traveling repairman problem with profits. In: Proceedings of the 10th Workshop on Algorithmic Approaches for Transportation Modelling, Optimization, and Systems, ATMOS 2010, pp. 34–44.

Dongarra, J.J., 2011. Performance of Various Computers using Standard Linear Equations Software. Tech. Rep. CS-89-85. Computer Science Department, University of Tennessee, Knoxville, TN, USA.

Ezzine, I.O., Semet, F., Chabchoub, H., 2010. New formulations for the traveling repairman problem. In: Proceedings of the 8th International Conference of Modeling and Simulation.

Fakcharoenphol, J., Harrelson, C., Rao, S., 2007. The k-traveling repairmen problem. ACM Transactions on Algorithms 3 (4), 40+.

Feo, T.A., Resende, M.G.C., 1995. Greedy randomized adaptive search procedures. Journal of Global Optimization 6 (2), 109–133.

Fischetti, M., Laporte, G., Martello, S., 1993. The delivery man problem and cumulative matroids. Operations Research 41 (6), 1055–1064.

García, A., Jodrá, P., Tejel, J., 2002. A note on the traveling repairman problem. Networks 40 (1), 27–31.

Goemans, M.X., Kleinberg, J.M., 1998. An improved approximation ratio for the minimum latency problem. Mathematical Programming 82 (1), 111–124.

Gouveia, L., Voss, S., 1995. A classification of formulations for the (time-dependent) traveling salesman problem. European Journal of Operational Research 83 (1), 69–82.

Heilporn, G., Cordeau, J.-F., Laporte, G., 2010. The delivery man problem with time windows. Discrete Optimization 7 (4), 269–282.

Kindervater, G., Savelsbergh, M., 1997. Vehicle routing: handling edge exchanges. In: Aarts, E., Lenstra, J. (Eds.), Local Search in Combinatorial Optimization. Wiley, New York, pp. 337–360.

Lourenço, H., Martin, O., Stützle, T., 2003. Iterated local search. In: Glover, F., Kochenberger, G. (Eds.), Handbook of Metaheuristics. Kluwer Academic Publishers, pp. 320–353.

Lucena, A., 1990. time-dependent traveling salesman problem – the deliveryman case. Networks 20 (6), 753–763.

Martin, O., Otto, S.W., Felten, E.W., 1991. Large-step Markov chains for the traveling salesman problem. Complex Systems 5 (3), 299–326.

Méndez-Díaz, I., Zabala, P., Lucena, A., 2008. A new formulation for the traveling deliveryman problem. Discrete Applied Mathematics 156 (17), 3223–3237.

Mladenović, N., Hansen, P., 1997. Variable neighborhood search. Computers & Operations Research 24 (11), 1097–1100.

Nagarajan, V., Ravi, R., 2008. The directed minimum latency problem. In: Proceedings of the 11th International Workshop, APPROX 2008, and 12th International Workshop, RANDOM 2008 on Approximation, Randomization and Combinatorial Optimization: Algorithms and Techniques, pp. 193–206.

Ngueveu, S., Prins, C., Wolfler Calvo, R., 2010. An effective memetic algorithm for the cumulative capacitated vehicle routing problem. Computers & Operations Research 37 (11), 1877–1885.

Picard, J.-C., Queyranne, M., 1978. The time-dependent traveling salesman problem and its application to the tardiness problem in one-machine scheduling. Operations Research 26 (1), 86–110.

Reinelt, G., 1991. TSPLIB – a traveling salesman problem library. INFORMS Journal on Computing 3 (4), 376–384.

Ribeiro, G., Laporte, G., 2012. An adaptive large neighborhood search heuristic for the cumulative capacitated vehicle routing problem. Computers & Operations Research 39 (3), 728–735.

Sahni, S., Gonzalez, T., 1976. P-complete approximation problems. Journal of the ACM 23 (3), 555–565.

Salehipour, A., Sörensen, K., Goos, P., Bräysy, O., 2011. Efficient GRASP + VND and GRASP + VNS metaheuristics for the traveling repairman problem. 4OR: A Quarterly Journal of Operations Research 9 (2), 189–209.

Savelsbergh, M., 1985. Local search in routing problems with time windows. Annals of Operations Research 4 (1), 285–305.

Sitters, R., 2002. The minimum latency problem is NP-hard for weighted trees. In: Proceedings of the 9th International Conference on Integer Programming and Combinatorial Optimization, IPCO 2002, pp. 230–239.

Subramanian, A., Drummond, L.M.A., Bentes, C., Ochi, L.S., Farias, R., 2010. A parallel heuristic for the vehicle routing problem with simultaneous pickup and delivery. Computers & Operations Research 37 (11), 1899–1911.

Tsitsiklis, J.N., 1992. Special cases of traveling salesman and repairman problems with time windows. Networks 22 (3), 263–282.

Van Eijl, C.A., 1995. A Polyhedral Approach to the Delivery Man problem. Tech. Rep. COSOR 95-19. Eindhoven University of Technology.

Vidal, T., Crainic, T., Gendreau, M., Prins, C., 2011. A Unifying View on Timing Problems and Algorithms. Tech. Rep. CIRRELT.

Wu, B.Y., Huang, Z.-N., Zhan, F.-J., 2004. Exact algorithms for the minimum latency problem. Information Processing Letters 92 (6), 303–309.